

Here are **clean, exam-oriented and interview-focused notes on Microservices in Advanced Java**. This is perfect for your Java exam, interviews, and Spring Boot microservices projects.

MICROSERVICES – ADVANCED JAVA (EXAM-ORIENTED NOTES)

1. What are Microservices? (Definition)

Microservices is an **architectural style** where a large application is built as a **collection of small, independent services**.

Each service is:

- Loosely coupled
- Independently deployable
- Runs in its own process
- Communicates via lightweight protocols (HTTP/REST, Messaging, gRPC)

Exam Line:

"Microservices break a monolithic application into independent components, each focusing on a specific business capability."

2. Why Microservices? (Advantages)

a) Independent Deployment

One service can be updated without affecting others.

b) Scalability

Each service can scale separately (Horizontal scaling).

c) Technology Flexibility

Different services can use:

- Java
- Node.js

- Python
- Database of choice

d) Fault Isolation

If one service fails, the whole system does not crash.

e) Faster Development

Teams build and deploy independently → CI/CD friendly.

3. Challenges of Microservices

Most asked in interviews:

a) Distributed System Complexity

Harder to manage than monolithic apps.

b) Network Latency

Since services communicate over network → slower than in-memory calls.

c) Data Consistency Issues

Database per service → difficult transactions.

d) Monitoring & Logging becomes complex

Need centralized tools.

4. Key Concepts in Java Microservices

4.1 Service Discovery (Eureka Server)

- Services register themselves.
- Other services discover them dynamically.
- Avoids hardcoding IPs.

Important terms:

- **Eureka Server** – registry
 - **Eureka Client** – service that registers
 - **Ribbon** – client-side load balancing
-

4.2 API Gateway (Spring Cloud Gateway)

Single entry point for all microservices.

Functions:

- Routing
 - Load Balancing
 - Authentication
 - Rate limiting
-

4.3 Load Balancing

Used to distribute incoming traffic across multiple instances.

Types:

- Client-side load balancing (Ribbon)
 - Server-side load balancing (NGINX, HAProxy)
-

4.4 Inter-Service Communication

a) REST Template – (Old)

b) Feign Client – (Modern way)

- Declarative REST client
- Easily call another microservice

Example:

java

 Copy code

```
@FeignClient(name = "payment-service")
public interface PaymentFeign {
    @GetMapping("/payment/status/{id}")
    String getPaymentStatus(@PathVariable int id);
}
```

4.5 Centralized Configuration (Config Server)

Store config files in Git and refresh without restarting.

4.6 Distributed Tracing (Zipkin / Sleuth)

Helps track requests across multiple services.

4.7 Circuit Breaker Pattern (Resilience4j / Hystrix)

Used to handle failure gracefully.

Example Use Case:

If Payment Service is down → return fallback response.

Example:

java

 Copy code

```
@CircuitBreaker(name = "payment", fallbackMethod = "fallbackPayment")
public String callPayment() {
    return paymentFeign.getPaymentStatus(1);
}

public String fallbackPayment(Exception e) {
    return "Payment Service is currently unavailable!";
}
```

5. Microservices Architecture Diagram (Exam ready)

Client → API Gateway → Service Discovery → Individual Microservices → Database per service → Message Broker → Monitoring

6. Microservices vs Monolithic Architecture

Feature	Monolithic	Microservices
Deployment	Single unit	Independent units
Database	One DB	DB Per Service

Feature	Monolithic	Microservices
Scalability	Difficult	Easy
Technology	Single tech stack	Multiple allowed
Fault Isolation	Weak	Strong

Most common exam question – always asked.

7. Important Tools & Technologies

a) Spring Boot → Used to build microservices

b) Spring Cloud → Provides:

- Eureka
- Config Server
- Gateway
- Feign
- Resilience4J
- Sleuth/Zipkin

c) Docker & Kubernetes (K8s)

For containerization and orchestration.

d) Message Brokers

- Kafka
- RabbitMQ

Used for asynchronous communication.

8. Code Flow of a Basic Microservice

Step 1: Create Spring Boot Application

- Add Web, JPA, DevTools

Step 2: Create REST Controller

java

 Copy code

```
@RestController
public class UserController {
    @GetMapping("/users")
    public String getUsers() {
        return "User Service is Running!";
    }
}
```

Step 3: Run on Port 8081

application.properties

properties

 Copy code

```
server.port=8081
spring.application.name=user-service
```

Step 4: Register with Eureka

Add dependencies:

- Eureka Client
- Spring Boot Actuator

Enable discovery:

java

 Copy code

```
@EnableDiscoveryClient
@SpringBootApplication
public class UserServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(UserServiceApplication.class, args);
    }
}
```

9. Interview Questions (Most Asked)

Q1. What is a microservice?

Small service focusing on a single functionality.

Q2. Why microservices?

Independent deployment, scalability, fault tolerance.

Q3. What is service discovery?

Dynamic detection of services using Eureka.

Q4. How do services communicate?

REST, Feign, Messaging.

Q5. What is circuit breaker?

Prevents cascading failures.

Q6. What is API Gateway?

Single entry point for routing, security, logging.

Q7. What is distributed tracing?

Tracking requests across multiple services.

If you want, I can also give:

- ✓ Microservices full architecture diagram
- ✓ Spring Boot + Eureka + Gateway full project example
- ✓ Microservices 10 marks answers
- ✓ Microservices MCQs for exam

Just tell me!



Introduction to Microservices
□ Microservices Architecture

Below is a clear, concise, exam-oriented explanation of:

- ✓ Introduction to Microservices
- ✓ Microservices Architecture

Exactly suitable for **Advanced Java exams and interviews.**

1. Introduction to Microservices (Exam-Oriented Answer)

Microservices is an architectural style in which a large application is built as a **collection of small, loosely coupled, independently deployable services**, where each service focuses on a **single business capability**.

In traditional Monolithic applications, all functionalities are tightly packed into one codebase. In contrast, Microservices break the application into smaller modules that can work, scale, and be deployed individually.

Key Characteristics

- Independent Deployment
- Loose Coupling
- Single Responsibility per service
- Own database per service
- Lightweight communication (REST/HTTP, Messaging)

Simple Exam Definition:

"Microservices is an architectural approach where an application is divided into small, autonomous services, each performing one function and communicating through lightweight protocols."

2. Microservices Architecture (Exam-Oriented Notes)

Microservices Architecture is the design pattern that structures an application as a set of **small, independent, modular services**. Each service can be developed, tested, deployed, updated, and scaled independently.

It solves the limitations of Monolithic architecture by providing **flexibility, scalability, and fault isolation**.

2.1 How Microservices Architecture Works

The architecture consists of multiple standalone services such as:

- User Service
- Order Service
- Payment Service

- **Inventory Service**

Each service:

1. Has its **own codebase**
 2. Runs in **separate process/container**
 3. Has its **own database**
 4. Communicates through **REST APIs or message queues**
 5. Can be deployed independently
-

2.2 Components of Microservices Architecture

1. API Gateway

- Single entry point for clients
- Handles routing, authentication, rate limiting

2. Service Discovery (Eureka Server)

- Keeps a registry of services
- Helps services find each other dynamically

3. Config Server

- Central place for configuration
- Updates without redeployment

4. Individual Microservices

Each with:

- Its own business logic
- Own database (MySQL, MongoDB, PostgreSQL)
- Independent deployment lifecycle

5. Communication Mechanisms

- **Synchronous:** REST, gRPC
- **Asynchronous:** Kafka, RabbitMQ

6. Load Balancer

- Distributes requests across multiple instances

7. Monitoring & Logging

- Tools like Prometheus, Grafana, Zipkin, Sleuth

8. Containerization

- Docker for packaging
- Kubernetes for deployment and scaling

2.3 Features of Microservices Architecture

✓ Scalability

Scale only the required service (e.g., Payment Service).

✓ Technology Diversity

Different services can use different languages/DBs.

✓ Fault Isolation

Failure of one service does not crash the whole system.

✓ Independent Deployment

Make changes without redeploying entire application.

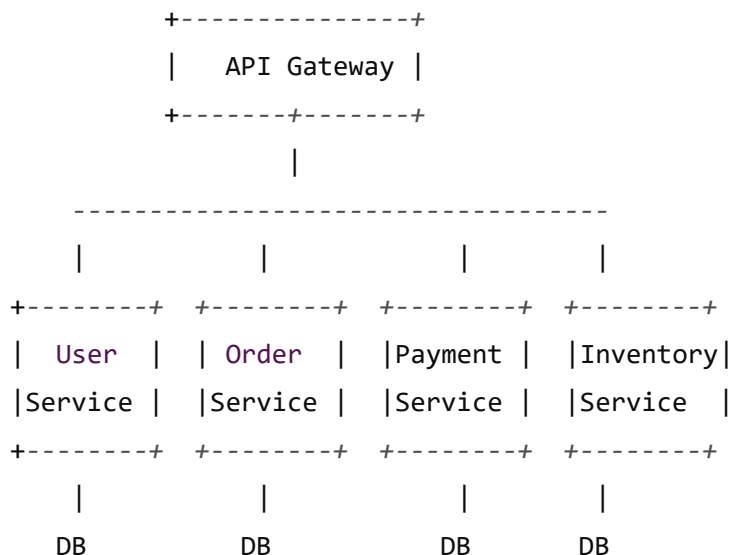
✓ CI/CD Friendly

Supports DevOps practices.

2.4 Microservices Architecture Diagram (Simple and Exam-Ready)

sql

 Copy code



2.5 Microservices Architecture VS Monolithic Architecture

Feature	Monolithic	Microservices
Deployment	Single unit	Independent services
Scalability	Whole app	Specific services
Fault Isolation	Poor	Strong
Technology	One stack	Multiple allowed
Database	Single DB	DB per service

2.6 Real-World Examples of Microservices

- **Netflix** → Streaming, user profile, recommendation services
- **Amazon** → Cart, payment, ordering, shipping services
- **Uber** → Maps, payment, trip management

If you want, I can also provide:

- ✂ Microservices Architecture 10-marks answer
- ✂ Diagram-based explanation
- ✂ Microservices vs SOA
- ✂ Full Microservices notes (20+ topics)

Just tell me!



Fragmentation of business requirement
□ Deployment pattern

Below are **clear, exam-oriented explanations** for:

- ✓ **Fragmentation of Business Requirements in Microservices**
- ✓ **Deployment Patterns in Microservices**

These answers are perfect for **Advanced Java, Spring Boot, and Microservices** exams/interviews.

1. Fragmentation of Business Requirement (Exam-Oriented)

Fragmentation of Business Requirements means **breaking down a large business requirement into smaller, manageable, independent functional units**.

This is the foundation of microservices design.

In Microservices Architecture, each service is built around a **specific business capability**, not technical layers.

Why Fragmentation is Needed?

- To avoid monolithic complexity
 - To support independent development
 - To isolate failures
 - To scale services independently
-

How Fragmentation Happens?

Business requirements are analysed and divided into **bounded contexts** (from Domain-Driven Design).

Example: E-Commerce System Requirement

Original Requirement:

"Customer places an order, makes payment, and checks delivery status."

Fragmentation into Microservices:

- 1. Customer Service**
Handles registration, login, customer details.
- 2. Product Catalog Service**
Shows product list, categories, pricing.

3. Order Service

Creates and manages orders.

4. Payment Service

Processes payments.

5. Inventory Service

Manages stock levels.

6. Delivery/Shipping Service

Tracks shipment and delivery.

Key Principles for Fragmentation

- **Single Responsibility Principle (SRP)**
Each microservice should do only one task.
- **Business Capability Alignment**
Fragment based on *what the business does*, not layers.
- **Bounded Contexts**
Each service manages its own domain and data.
- **Loose Coupling**
Services should not depend on each other tightly.

Exam Definition:

"Fragmentation of business requirements is the process of dividing a large business problem into smaller, domain-specific microservices that can operate, deploy, and scale independently."

2. Deployment Patterns in Microservices (Exam-Oriented)

Microservices support many deployment patterns based on the architecture, scalability, availability, and DevOps strategy.

Below are the most **important and exam-relevant patterns**:

2.1 Single Service Instance per Host

Each microservice runs on a **dedicated virtual machine or physical server**.

Pros:

- Simple
- Good isolation

Cons:

- High cost
 - Hard to scale
-

2.2 Multiple Service Instances per Host

Multiple microservices run inside the same physical server/VM.

Pros:

- Better resource utilization
- Cost-efficient

Cons:

- Fault isolation is weaker
-

2.3 Service Instance per Container (Most common with Docker)

Each microservice is packaged and deployed inside a **Docker container**.

Pros:

- Portable and lightweight
- Reliable deployments
- Easy scaling

Cons:

- Requires container management
-

2.4 Service Instance per VM

Each service runs inside its own **virtual machine**.

Pros:

- Strong isolation
- Easy monitoring

Cons:

- Slower deployment
 - Expensive
-

2.5 Serverless Deployment (AWS Lambda / Azure Functions)

Microservices run as small serverless functions.

Pros:

- Auto-scaling
- Extremely low operational overhead

Cons:

- Cold start issues
 - Vendor lock-in
-

2.6 Blue-Green Deployment (Used in CI/CD)

Two identical production environments:

- **Blue** → current version
- **Green** → new version

Traffic switches to "green" after testing.

Benefits:

- Zero downtime
 - Easy rollback
-

2.7 Canary Deployment

Release new version to a **small percentage of users** first.

If successful → rollout to everyone.

Benefits:

- Safe testing
- Controlled rollout

2.8 Rolling Deployment

Update services **one instance at a time** while others continue running.

Benefits:

- No downtime
- Smooth upgrade

2.9 Sidecar Deployment

Used in Kubernetes/Service Meshes.

A **sidecar container** runs beside the main service for:

- Logging
- Monitoring
- Configurations
- Network proxy (Istio/Envoy)

2.10 Ambassador Deployment

A helper process that communicates on behalf of the service to external systems (e.g., API gateways, proxies).

Exam-Ready Summary: Deployment Patterns

Pattern	Description
Single Instance per Host	One service per server
Multiple Instances per Host	Multiple services share a server
Instance per Container	Each service in a Docker container
Instance per VM	Each service in a virtual machine

Pattern	Description
Serverless	Service runs as functions
Blue-Green	Two environments; switch traffic
Canary	Release to small segment first
Rolling Update	Update instances gradually
Sidecar	Helper container for logs, monitoring
Ambassador	Handles external communication

If you want, I can also provide:

- ✂ Microservices Architecture Diagram
- ✂ Exam 10-marks answer
- ✂ Microservices full notes (20+ topics)
- ✂ Spring Boot microservices project flow

Just tell me!



API gateway
□ Service Discovery

Below are clear, exam-oriented, and interview-ready explanations for:

- ✓ API Gateway
- ✓ Service Discovery

Perfect for **Advanced Java / Spring Boot Microservices** exams.

1. API Gateway (Exam-Oriented Explanation)

An **API Gateway** is a server that acts as a **single entry point** for all client requests in a microservices architecture.

Instead of calling microservices directly, clients send all requests to the gateway, and it **routes** them to the appropriate microservice.

It simplifies client communication and provides cross-cutting features.

1.1 Why API Gateway Is Needed?

- Microservices run on different ports/servers
 - Exposing all services to clients becomes complex
 - Authentication, rate limiting, logging need centralization
-

1.2 Responsibilities of API Gateway

1. Routing

Forward requests to correct microservice.

2. Load Balancing

Distribute requests across multiple service instances.

3. Authentication & Authorization

Validates tokens (JWT/OAuth).

4. Centralized Logging

Tracks all incoming/outgoing requests.

5. Rate Limiting

Controls number of requests per time window.

6. Caching

Improves performance by storing common responses.

7. Request/Response Transformation

Modify headers, URL paths, etc.

1.3 API Gateway in Spring Boot

Most commonly used:

- Spring Cloud Gateway
- Netflix Zuul (old)

Example Routing (Spring Cloud Gateway):

yaml

 Copy code

```
spring:
  cloud:
    gateway:
      routes:
        - id: user-service
          uri: http://localhost:8081
          predicates:
            - Path=/user/**
```

1.4 Advantages of API Gateway

- Hides internal service structure
- Reduces complexity for clients
- Enhances security
- Improves performance
- Supports monitoring

1.5 Simple Exam Definition

"API Gateway is a single entry point for all client requests that handles routing, security, load balancing, and monitoring in a microservices architecture."

2. Service Discovery (Exam-Oriented Explanation)

Service Discovery is the mechanism for automatically detecting and locating microservices on a network.

Microservices frequently change their IPs and ports (due to auto-scaling, restarts, containers).

Service Discovery helps services find each other dynamically.

2.1 Why Service Discovery?

- Microservices run on dynamic hosts
- Hard-coding IPs is not possible

- Ensures reliable communication
-

2.2 Types of Service Discovery

1. Client-Side Discovery

Client finds the service instance from a **Service Registry** (like Netflix Eureka).

Client decides which instance to call.

Example: Netflix Ribbon

Process:

Client → Queries Eureka → Gets instance list → Loads balances → Calls an instance

2. Server-Side Discovery

The **API Gateway** queries the Service Registry and routes traffic.

Process:

Client → API Gateway → Service Registry → Target service

Used by Kubernetes, AWS ELB.

2.3 Service Registry

Central database that stores:

- Service name
- Service URL
- Health status

Examples:

- Netflix Eureka
 - Consul
 - Zookeeper
 - Kubernetes API Server (built-in)
-

2.4 Eureka Server (Spring Boot)

Most common in Java microservices.

Components:

1. Eureka Server → Registry
2. Eureka Client → Services registering themselves

2.5 How Service Discovery Works (Eureka Example)

Step 1: Microservice starts

Step 2: Registers itself with Eureka

Step 3: Other microservices retrieve its location via Eureka

Step 4: Communication happens dynamically

2.6 Eureka Client Example

Main Class:

java

 Copy code

```
@EnableDiscoveryClient
@SpringBootApplication
public class UserServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(UserServiceApplication.class, args);
    }
}
```

Properties:

properties

 Copy code

```
spring.application.name=user-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

2.7 Advantages of Service Discovery

- No hardcoding of URLs
 - Auto-scaling friendly
 - Fault-tolerant
 - Supports dynamic updates
 - Easier load balancing
-

2.8 Simple Exam Definition

"Service Discovery is a mechanism where microservices automatically register and discover each other using a Service Registry like Eureka, enabling dynamic communication without hard-coded URLs."

If you want, I can also provide:

- ✦ API Gateway + Eureka architecture diagram
- ✦ 10-marks answer combining both topics
- ✦ Full microservices notes for exam
- ✦ Spring Boot project (Gateway + Eureka + 2 services)

Just tell me!



Database Management for Microservices

Below is a **clear, exam-oriented and interview-focused explanation** of **Database Management for Microservices**.

Perfect for **Advanced Java, Spring Boot Microservices, and architecture exams**.

Database Management for Microservices (Exam-Oriented Notes)

In Microservices Architecture, each service is **independent**, and therefore database management becomes a critical part of design.

Microservices follow “**Database per Service**” principle, meaning **each microservice has its own data storage**, ensuring loose coupling and high autonomy.

1. Why Separate Databases for Microservices?

a) Loose Coupling

Services don't depend on each other's schema.

b) Independence

Services can:

- Deploy independently
- Change database structure independently
- Scale independently

c) Technology Flexibility

One service can use:

- PostgreSQL
- Another MongoDB
- Another Redis

d) Fault Isolation

Database failure in one service does not crash the entire system.

2. Database Patterns in Microservices

Here are the **most important**, exam-relevant database patterns:

2.1 Database per Service (MOST IMPORTANT)

Each microservice owns its own private database.

Benefits:

- Maximum isolation
- Easy to scale
- Independent schema

- High autonomy

Example:

- Order Service → orders_db
 - Payment Service → payment_db
 - Inventory Service → inventory_db
-

2.2 Shared Database (Not Recommended but used in legacy)

Multiple microservices use a **single shared database**.

Problems:

- Tight coupling
- Database bottleneck
- Difficult schema changes
- Breaks microservice principles

Used only for:

- Legacy migration
 - Temporary integration
-

2.3 Database per Service with Shared Schema

Each service has its own tables but stored in one DB instance.

Example:

Same DB → Multiple schemas or tables grouped by service.

Used when:

- Hardware limited
 - Small projects
-

3. Handling Data Consistency in Microservices

Separate databases cause **distributed systems challenges**.

3.1 Types of Consistency

a) Strong Consistency

All services see the same data immediately.

⚠️ Not possible easily in microservices.

b) Eventual Consistency (Most Used)

Data becomes consistent after a short delay.

4. Data Communication Patterns Between Services

4.1 Synchronous Communication

Using REST API

- Fast
 - Real-time
 - But tightly coupled
-

4.2 Asynchronous Communication (Recommended)

Using events through:

- Kafka
- RabbitMQ
- ActiveMQ

Example:

Order Service → sends "Order Created" Event

Payment Service → listens and processes payment

5. Database Transaction Management in Microservices

Microservices **cannot** use traditional ACID transactions across multiple DBs.

Solution: Distributed Transactions Patterns

5.1 Saga Pattern (MOST IMPORTANT)

A sequence of local transactions where each step publishes an event.

Two types:

a) Choreography Saga

- No central controller
- Each service listens to events
- Best for smaller workflows

b) Orchestration Saga

- One central coordinator (Orchestrator Service)
 - Orchestrator calls each service step-by-step
-

5.2 Two-Phase Commit (2PC)

A coordinator prepares and commits transactions.

⚠ Not recommended for microservices

- Slow
 - Blocking
 - Not fault-tolerant
-

6. Database Replication & Scalability

Microservices support:

a) Horizontal Scaling

- Duplicate service instances
- Each with replicated DB or shared cluster

b) Read/Write Separation

- Master DB → Write
- Replica DB → Read

Helps reduce load.

7. Polyglot Persistence (Important Concept)

Microservices allow using **different database types** based on service needs.

Examples:

- Product Service → Elasticsearch (search optimized)

- Payment Service → PostgreSQL (transactional)
 - Notification Service → Redis (fast caching)
-

8. Data Migration in Microservices

Each service must manage its own migrations using tools like:

- Flyway
- Liquibase

These tools ensure DB changes are version controlled.

9. Security in Microservices Databases

Each service should have:

- Its own DB credentials
 - Encrypted credentials (Spring Vault)
 - Role-based access
 - Network isolation (VPC/Subnets)
-

10. Summary (Exam-Ready Points)

Database Management in Microservices includes:

- ✓ Database per service
 - ✓ Eventual consistency
 - ✓ Saga pattern
 - ✓ Polyglot persistence
 - ✓ Asynchronous communication
 - ✓ Data replication and scaling
 - ✓ Independent schema migration
-

If you want, I can also give:

- ✦ Microservices Database Architecture Diagram
- ✦ 10-Marks long answer

👉 Saga Pattern detailed explanation with example

